

PySpark from scratch

● LEVEL 1 — DataFrame Basics

1. Read the employees table into a DataFrame.

```
df_emp = spark.table("ashish.employee.employees")
```

LinkedIn@AshishZope

2. Display the complete employee DataFrame.

```
display(df_emp)  
df_emp.show(10) # 10 in the show() method is the number of rows to display
```

LinkedIn@AshishZope

3. Display the first 10 employees.

```
display(df_emp.limit(10))
```

LinkedIn@AshishZope

4. Display the first 20 employees without truncating string values.

```
df_emp.show(20, truncate=False)
```

LinkedIn@AshishZope

5. Print the schema of the employee DataFrame.

```
df_emp.schema  
df_emp.printSchema()
```

LinkedIn@AshishZope

6. Print all column names.

LinkedIn@AshishZope

```
print(df_emp.columns)
df_emp.columns
```

7. Count the total number of employees.

LinkedIn@AshishZope

```
df_emp.count()
```

8. Display only the first employee.

LinkedIn@AshishZope

```
display(df_emp.limit(1))
```

9. Display 5 employees using `take()`.

LinkedIn@AshishZope

```
df_emp.take(5)
```

10. Check the number of partitions of the DataFrame.

LinkedIn@AshishZope

```
df_emp.rdd.getNumPartitions()
```

11. Display employees using `display()`.

LinkedIn@AshishZope

```
display(df_emp)
```

12. Display employees using `show()`.

LinkedIn@AshishZope

```
df_emp.show()
```

13. Display the DataFrame vertically.

LinkedIn@AshishZope

```
df_emp.show(vertical=True)
```

14. Create a temporary view called `employees_view`.

LinkedIn@AshishZope

```
df_emp.createOrReplaceTempView("employees_view")
```

15. Read the temporary view using Spark SQL.

LinkedIn@AshishZope

```
df_emp_sql = spark.sql("SELECT * FROM employees_view")
```

● LEVEL 2 — SELECT Operations

16. Select only: `employee_id`, `first_name`, `last_name`, `salary`

LinkedIn@AshishZope

```
df_emp.select("employee_id", "first_name", "last_name", "salary").show()
```

17. Select employee ID, name and department.

LinkedIn@AshishZope

```
df_emp.join(df_dept, df_emp.department_id == df_dept.department_id,
"inner").select("employee_id", "first_name", "last_name",
"department_name").show()
```

18. Select employee ID, email and phone.

LinkedIn@AshishZope

```
display(df_emp.select("employee_id", "email", "phone"))
```

19. Rename **first_name** as **employee_first_name**.

LinkedIn@AshishZope

```
df_emp.selectExpr("first_name AS employee_first_name").show()
```

20. Rename **salary** as **annual_salary**.

LinkedIn@AshishZope

```
df_emp.selectExpr("salary AS annual_salary").show()
```

21. Select salary and bonus.

LinkedIn@AshishZope

```
df_emp.select("salary", "bonus").show()
```

22. Select employee information and rename multiple columns.

LinkedIn@AshishZope

```
df_emp.selectExpr(  
    "first_name || ' ' || last_name as employee_full_name",  
    "salary + bonus as total_compensation",  
    "salary + bonus > 1000000 as is_highly_paid"  
) .show()
```

23. Select all columns except **phone**.

LinkedIn@AshishZope

```
df_emp.drop("phone").show()
```

24. Select only: **text**, **employee_id**, **first_name**, **last_name**, **city**, **state**

LinkedIn@AshishZope

```
df_emp.select("employee_id", "first_name", "last_name", "city",
"state").show()
```

25. Select employee information and create an alias for salary.

LinkedIn@AshishZope

```
df_emp.selectExpr("first_name AS f_name", "salary AS salary_new").show()
```



LEVEL 3 — FILTER Operations

26. Find all employees whose salary is greater than 1,000,000.

LinkedIn@AshishZope

```
df_emp.filter(df_emp.salary > 1000000).show()
df_emp.select("employee_id", "first_name", "last_name", "city", "state",
"salary").filter("salary > 1000000").show()
```

27. Find employees whose salary is less than 700,000.

LinkedIn@AshishZope

```
df_emp.filter(df_emp.salary < 700000).show()
```

28. Find employees whose salary is between 700,000 and 1,500,000.

LinkedIn@AshishZope

```
df_emp.filter((df_emp.salary >= 700000) & (df_emp.salary <= 1500000)).show()
df_emp.filter("salary > 700000 AND salary < 1500000").show()
```

29. Find employees from Pune.

LinkedIn@AshishZope

```
df_emp.filter(df_emp.city == "Pune").show()
```

30. Find employees from Mumbai.

LinkedIn@AshishZope

```
df_emp.filter(df_emp.city == "Mumbai").show()
```

31. Find employees from Pune or Mumbai.

LinkedIn@AshishZope

```
df_emp.filter((df_emp.city == "Pune") | (df_emp.city == "Mumbai")).show()  
df_emp.filter("city == 'Mumbai' OR city == 'Pune'").show()  
df_emp.filter("city IN ('Mumbai', 'Pune')").show()
```

32. Find employees who are currently active.

LinkedIn@AshishZope

```
df_emp.filter(df_emp.employment_status == "Active").show()
```

33. Find inactive employees.

LinkedIn@AshishZope

```
df_emp.filter(df_emp.employment_status == "Inactive").show()
```

34. Find employees whose gender is Male.

LinkedIn@AshishZope

```
df_emp.filter(df_emp.gender == "Male").show()
```

35. Find employees whose salary is greater than 1,000,000 and who are active.

LinkedIn@AshishZope

```
df_emp.filter((df_emp.salary > 1000000) & (df_emp.employment_status ==
"Active")).show()
df_emp.filter("salary > 1000000 AND employment_status == 'Active'").show()
```

36. Find employees from Pune with salary greater than 1,000,000.

LinkedIn@AshishZope

```
df_emp.filter((df_emp.city == "Pune") & (df_emp.salary > 1000000)).show()
df_emp.filter("salary > 1000000 AND city == 'Pune'").show()
```

37. Find employees who are not from Pune.

LinkedIn@AshishZope

```
df_emp.filter(df_emp.city != "Pune").show()
df_emp.filter("city != 'Pune'").show()
```

38. Find employees whose department ID is 1.

LinkedIn@AshishZope

```
df_emp.filter(df_emp.department_id == 1).show()
```

39. Find employees whose job ID is 2 or 3.

LinkedIn@AshishZope

```
df_emp.filter((df_emp.job_id == 2) | (df_emp.job_id == 3)).show()
df_emp.filter("job_id IN (2, 3)").show()
df_emp.filter(df_emp.job_id.isin(2,3)).show()
```

40. Find employees whose manager ID is NULL.

LinkedIn@AshishZope

```
df_emp.filter(df_emp.manager_id.isNull()).show()
```

● LEVEL 4 — Computed Columns

41. Create **annual_salary** from salary.

LinkedIn@AshishZope

```
df_emp.withColumn("annual_salary", df_emp.salary * 12).show()
```

42. Create **increased_salary** with a 10% increment.

LinkedIn@AshishZope

```
df_emp.withColumn(  
    "salary_10_percent",  
    col("salary") * 10 / 100  
)  
.select(  
    col("first_name").alias("f_name"),  
    col("salary"),  
    col("salary_10_percent").alias("increased_salary")  
)  
.show()
```

43. Create **bonus_amount** as 15% of salary.

DATA ENGINEER

LinkedIn@AshishZope

```
df_emp.withColumn("bonus_amount", df_emp.salary * 15 / 100).select(  
    col("first_name").alias("f_name"),  
    col("salary"),  
    col("bonus_amount").alias("bonus").cast("int")  
)  
.show()
```

44. Create **total_compensation: salary + bonus**

LinkedIn@AshishZope

```
df_emp.withColumn("total_compensation", df_emp.salary + df_emp.bonus).select(
    col("first_name").alias("f_name"),
    col("salary"),
    col("bonus"),
    col("total_compensation")
).show()
```

45. Create **double_salary**.

LinkedIn@AshishZope

```
df_emp.withColumn("double_salary", col("salary") * 2).select("first_name",
"double_salary").show(5)
```

46. Create **salary_after_5_percent_increment**.

LinkedIn@AshishZope

```
df_emp.withColumn("salary_after_5_percent_increment", col("salary") *
1.05).select("first_name", "salary_after_5_percent_increment").show(5)
```

47. Create **salary_difference** between salary and bonus.

LinkedIn@AshishZope

```
df_emp.withColumn("salary_difference", col("salary") -
col("bonus")).select("first_name", "salary", "bonus",
"salary_difference").show(5)
```

48. Create **monthly_salary** from annual salary.

LinkedIn@AshishZope

```
df_emp.withColumn("monthly_salary", col("salary") / 12) \
    .select("first_name", col("monthly_salary").cast("int")) \
    .show(5)
```

49. Create **monthly_bonus**.

```
df_emp.withColumn("monthly_bonus", col("bonus") / 12) \
    .select("first_name", col("monthly_bonus").cast("int")) \
    .show(5)
```

50. Create salary_category:

```
< 700000      → Low
700000-1200000 → Medium
1200000-2000000 → High
> 2000000      → Very High
```

```
df_emp.selectExpr(
    "first_name",
    "salary",
    """CASE
        WHEN salary < 700000 THEN 'Low'
        WHEN salary < 1200000 THEN 'Medium'
        WHEN salary < 2000000 THEN 'High'
        ELSE 'Very High'
    END AS salary_category"""
).show(10)
```

51. Create employee_type based on employment status.

```
df_emp.selectExpr(
    "first_name",
    """CASE
        WHEN employment_status = 'Active' THEN 'Current Employee'
        WHEN employment_status = 'Inactive' THEN 'Former Employee'
        ELSE 'Unknown'
    END AS employee_type"""
).show()
```

52. Create a column indicating whether salary is above 1 million:

LinkedIn@AshishZope

Yes / No

LinkedIn@AshishZope

```
df_emp.selectExpr(  
    "first_name",  
    """CASE  
        WHEN salary > 1000000 THEN 'Yes'  
        ELSE 'No'  
    END AS salary_above_1_million""")  
)
```

53. Create **manager_status**:

LinkedIn@AshishZope

Manager Assigned
No Manager

LinkedIn@AshishZope

```
df_emp.selectExpr(  
    "first_name",  
    """CASE  
        WHEN manager_id IS NOT NULL THEN 'Manager Assigned'  
        ELSE 'No Manager'  
    END AS manager_status""")  
)
```

54. Create **full_name**.

LinkedIn@AshishZope

```
df_emp.selectExpr(  
    "first_name",  
    "last_name",  
    "concat(first_name, ' ', last_name) as full_name"  
)
```

55. Create **email_domain**.

```
df_emp.selectExpr(  
    "email",  
    "split(email, '@')[1] as email_domain"  
).show(5)
```

● LEVEL 5 — String Operations

56. Convert first names to uppercase.

```
df_emp.selectExpr("UPPER(first_name) AS first_name_upper").show(5)
```

57. Convert last names to lowercase.

```
df_emp.selectExpr("LOWER(last_name) AS last_name_lower").show(5)
```

58. Create full name using first and last name.

```
df_emp.withColumn(  
    "full_name",  
    concat_ws(" ", col("first_name"), col("last_name"))  
).select("first_name", "last_name", "full_name").show(5)
```

59. Extract the first 3 characters of first name.

```
df_emp.selectExpr("SUBSTRING(first_name, 1, 3) AS first_name_prefix").show(5)
```

60. Extract the last 3 characters of last name.

LinkedIn@AshishZope

```
df_emp.selectExpr("SUBSTRING(last_name, LENGTH(last_name) - 2, 3) AS  
last_name_suffix").show(5)
```

61. Find the length of employee names.

LinkedIn@AshishZope

```
df_emp.selectExpr("first_name", "LENGTH(first_name) AS  
first_name_length").show(5)
```

62. Remove spaces from employee names.

LinkedIn@AshishZope

```
df_emp.selectExpr("TRIM(first_name) AS clean_first_name").show(5)
```

63. Replace part of an email address.

LinkedIn@AshishZope

```
df_emp.selectExpr("REGEXP_REPLACE(email, '@gmail.com', '@company.com') AS  
updated_email").show(5)
```

64. Extract email domain.

LinkedIn@AshishZope

```
df_emp.selectExpr("SPLIT(email, '@')[1] AS email_domain").show(5)
```

65. Check whether email contains @.

LinkedIn@AshishZope

```
df_emp.selectExpr("email", "CONTAINS(email, '@') AS  
has_at_symbol").filter("has_at_symbol = true").show(5)
```

66. Find employees whose first name starts with A.

LinkedIn@AshishZope

```
df_emp.filter(col("first_name").startswith("A")).show(5)
```

67. Find employees whose last name ends with a.

LinkedIn@AshishZope

```
df_emp.filter(lower(col("last_name")).endswith("a")).show(5)
```

68. Create an employee username:

LinkedIn@AshishZope

```
firstname.lastname
```

LinkedIn@AshishZope

```
df_emp.withColumn(
    "username",
    concat_ws(".", lower(col("first_name")), lower(col("last_name")))
).select("first_name", "last_name", "username").show(5)
```

69. Create an employee code:

LinkedIn@AshishZope

```
EMP_<employee_id>
```

LinkedIn@AshishZope

```
df_emp.withColumn("employee_code", concat(lit("EMP_"),
col("employee_id"))).select("employee_id", "employee_code").show(5)
```

70. Create a display name:

LinkedIn@AshishZope

```
LASTNAME, FIRSTNAME
```

```
df_emp.withColumn(  
    "display_name",  
    concat_ws(" ", col("last_name"), col("first_name"))  
).select("last_name", "first_name", "display_name").show(5)
```

● LEVEL 6 — NULL Handling

71. Find employees whose manager ID is NULL.

```
df_emp.filter(col("manager_id").isNull()).show()
```

72. Count NULL values in every column.

```
from pyspark.sql.functions import col, when, count  
  
null_counts = df_emp.select([  
    count(when(col(c).isNull(), c)).alias(c) for c in df_emp.columns  
)  
null_counts.show()
```

73. Replace NULL city with Unknown.

```
df_emp.fillna({"city": "Unknown"}).show()
```

74. Replace NULL salary with 0.

```
df_emp.fillna({"salary": 0}).show()
```

75. Replace NULL bonus with 0.

LinkedIn@AshishZope

```
df_emp.fillna({"bonus": 0}).show()
```

76. Replace NULL manager ID with -1.

LinkedIn@AshishZope

```
df_emp.fillna({"manager_id": -1}).show()
```

77. Create a column showing whether manager ID is NULL.

LinkedIn@AshishZope

```
from pyspark.sql.functions import when

df_emp.withColumn(
    "manager_present",
    when(col("manager_id").isNull(), "No").otherwise("Yes")
).select("first_name", "manager_id", "manager_present").show(5)
```

78. Find employees with missing phone numbers.

LinkedIn@AshishZope

```
df_emp.filter(col("phone").isNull() | (trim(col("phone")) == "")).show()
```

79. Find employees with missing email addresses.

LinkedIn@AshishZope

```
df_emp.filter(col("email").isNull() | (trim(col("email")) == "")).show()
```

80. Remove rows where employee ID is NULL.

LinkedIn@AshishZope

```
df_emp.na.drop(subset=["employee_id"]).show()
```


● LEVEL 7 — Sorting

81. Sort employees by salary ascending.

```
df_emp.orderBy("salary").show(10)
```

LinkedIn@AshishZope

82. Sort employees by salary descending.

```
df_emp.orderBy(col("salary").desc()).show(10)
```

LinkedIn@AshishZope

83. Sort employees by first name.

```
df_emp.orderBy("first_name").show(10)
```

LinkedIn@AshishZope

84. Sort employees by department and salary.

```
df_emp.orderBy("department_id", "salary").show(10)
```

LinkedIn@AshishZope

85. Find the highest-paid employee.

```
df_emp.orderBy(col("salary").desc()).first()
```

LinkedIn@AshishZope

86. Find the lowest-paid employee.

LinkedIn@AshishZope

```
df_emp.orderBy(col("salary").asc()).first()
```

87. Find the top 5 highest-paid employees.

LinkedIn@AshishZope

```
df_emp.orderBy(col("salary").desc()).limit(5).show()
```

88. Find the bottom 5 employees by salary.

LinkedIn@AshishZope

```
df_emp.orderBy(col("salary").asc()).limit(5).show()
```

89. Sort by salary descending and bonus ascending.

LinkedIn@AshishZope

```
df_emp.orderBy(col("salary").desc(), col("bonus").asc()).show(10)
```

90. Sort employees by hire date.

LinkedIn@AshishZope

```
df_emp.orderBy(col("hire_date").asc()).show(10)
```



● LEVEL 8 — DISTINCT & DUPLICATES

91. Find distinct cities.

LinkedIn@AshishZope

```
df_emp.select("city").distinct().show()
```

92. Find distinct states.

LinkedIn@AshishZope

```
df_emp.select("state").distinct().show()
```

93. Find distinct departments.

LinkedIn@AshishZope

```
df_emp.select("department_id").distinct().show()
```

94. Count distinct cities.

LinkedIn@AshishZope

```
df_emp.select("city").distinct().count()
```

95. Count distinct departments.

LinkedIn@AshishZope

```
df_emp.select("department_id").distinct().count()
```

96. Find duplicate employee IDs.

LinkedIn@AshishZope

```
df_emp.groupBy("employee_id").count().filter("count > 1").show()
```

97. Find duplicate email addresses.

LinkedIn@AshishZope

```
df_emp.groupBy("email").count().filter("count > 1").show()
```

98. Remove duplicate employees based on employee ID.

```
df_emp.dropDuplicates(["employee_id"]).show()
```

99. Remove duplicate records based on email.

```
df_emp.dropDuplicates(["email"]).show()
```

100. Keep the latest record when duplicate employee IDs exist.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number

window_spec =
Window.partitionBy("employee_id").orderBy(col("hire_date").desc())
latest_emps = df_emp.withColumn("rn",
row_number().over(window_spec)).filter(col("rn") == 1).drop("rn")
latest_emps.show()
```



LEVEL 9 — Aggregations

101. Find total salary of all employees.

```
df_emp.agg(sum("salary").alias("total_salary")).show()
```

102. Find average salary.

```
df_emp.agg(avg("salary").alias("avg_salary")).show()
```

103. Find minimum salary.

LinkedIn@AshishZope

```
df_emp.agg(min("salary").alias("min_salary")).show()
```

104. Find maximum salary.

LinkedIn@AshishZope

```
df_emp.agg(max("salary").alias("max_salary")).show()
```

105. Find total bonus.

LinkedIn@AshishZope

```
df_emp.agg(sum("bonus").alias("total_bonus")).show()
```

106. Count employees.

LinkedIn@AshishZope

```
df_emp.count()
```

107. Count employees by department.

LinkedIn@AshishZope

```
df_emp.groupBy("department_id").count().show()
```

108. Find average salary by department.

LinkedIn@AshishZope

```
df_emp.groupBy("department_id").agg(avg("salary").alias("avg_salary")).show()
```

109. Find maximum salary by department.

LinkedIn@AshishZope

```
df_emp.groupBy("department_id").agg(max("salary").alias("max_salary")).show()
```

110. Find minimum salary by department.

LinkedIn@AshishZope

```
df_emp.groupBy("department_id").agg(min("salary").alias("min_salary")).show()
```

111. Find total salary by department.

LinkedIn@AshishZope

```
df_emp.groupBy("department_id").agg(sum("salary").alias("total_salary")).show()  
)
```

112. Find average bonus by department.

LinkedIn@AshishZope

```
df_emp.groupBy("department_id").agg(avg("bonus").alias("avg_bonus")).show()
```

113. Find employee count by city.

LinkedIn@AshishZope

```
df_emp.groupBy("city").count().show()
```

114. Find employee count by state.

LinkedIn@AshishZope

```
df_emp.groupBy("state").count().show()
```

115. Find average salary by city.

LinkedIn@AshishZope

```
df_emp.groupBy("city").agg(avg("salary").alias("avg_salary")).show()
```

116. Find departments having more than 5 employees.

LinkedIn@AshishZope

```
df_emp.groupBy("department_id").count().filter(col("count") > 5).show()
```

117. Find departments whose average salary is greater than 1 million.

LinkedIn@AshishZope

```
df_emp.groupBy("department_id").agg(avg("salary").alias("avg_salary")).filter(
col("avg_salary") > 1000000).show()
```

118. Find the department with the highest total salary.

LinkedIn@AshishZope

```
df_emp.groupBy("department_id").agg(sum("salary").alias("total_salary")).order
By(col("total_salary").desc()).limit(1).show()
```

119. Find the department with the highest average salary.

LinkedIn@AshishZope

```
df_emp.groupBy("department_id").agg(avg("salary").alias("avg_salary")).orderBy
(col("avg_salary").desc()).limit(1).show()
```

120. Find the city having the highest employee count.

LinkedIn@AshishZope

```
df_emp.groupBy("city").count().orderBy(col("count").desc()).limit(1).show()
```



LEVEL 10 — GROUP BY + HAVING **Scenarios**

121. Find departments with more than 3 employees.

LinkedIn@AshishZope

```
df_emp.groupBy("department_id").count().filter(col("count") > 3).show()
```

122. Find departments with average salary > ₹10 lakh.

LinkedIn@AshishZope

```
df_emp.groupBy("department_id").agg(avg("salary").alias("avg_salary")).filter(
col("avg_salary") > 1000000).show()
```

123. Find cities with more than 5 employees.

LinkedIn@AshishZope

```
df_emp.groupBy("city").count().filter(col("count") > 5).show()
```

124. Find departments where maximum salary > ₹20 lakh.

LinkedIn@AshishZope

```
df_emp.groupBy("department_id").agg(max("salary").alias("max_salary")).filter(
col("max_salary") > 2000000).show()
```

125. Find departments where total salary > ₹50 lakh.

LinkedIn@AshishZope

```
df_emp.groupBy("department_id").agg(sum("salary").alias("total_salary")).filter(
col("total_salary") > 5000000).show()
```

126. Find cities where average salary > ₹10 lakh.

LinkedIn@AshishZope

```
df_emp.groupBy("city").agg(avg("salary").alias("avg_salary")).filter(col("avg_
salary") > 1000000).show()
```


127. Find job IDs having more than 5 employees.

LinkedIn@AshishZope

```
df_emp.groupBy("job_id").count().filter(col("count") > 5).show()
```

128. Find departments having at least one employee earning > ₹20 lakh.

LinkedIn@AshishZope

```
df_emp.filter(col("salary") > 2000000).groupBy("department_id").count().show()
```

129. Find departments where minimum salary is > ₹5 lakh.

LinkedIn@AshishZope

```
df_emp.groupBy("department_id").agg(min("salary").alias("min_salary")).filter(
col("min_salary") > 500000).show()
```

130. Find departments with both:

LinkedIn@AshishZope

```
employee_count > 5
average_salary > 10 lakh
```

LinkedIn@AshishZope

```
df_emp.groupBy("department_id").agg(
    count("*").alias("employee_count"),
    avg("salary").alias("average_salary")
).filter((col("employee_count") > 5) & (col("average_salary") >
1000000)).show()
```



LEVEL 11 — Date Operations

131. Find employees hired after 2023-01-01.

LinkedIn@AshishZope

```
df_emp.filter(to_date(col("hire_date")) > lit("2023-01-01")).show()
```

132. Find employees hired before 2023-01-01.

LinkedIn@AshishZope

```
df_emp.filter(to_date(col("hire_date")) < lit("2023-01-01")).show()
```

133. Find employees hired during 2023.

LinkedIn@AshishZope

```
df_emp.filter(year(to_date(col("hire_date")))) == 2023).show()
```

134. Find employees hired during 2024.

LinkedIn@AshishZope

```
df_emp.filter(year(to_date(col("hire_date")))) == 2024).show()
```

135. Extract hire year.

LinkedIn@AshishZope

```
df_emp.withColumn("hire_year",  
year(to_date(col("hire_date")))).select("first_name", "hire_year").show(5)
```

136. Extract hire month.

LinkedIn@AshishZope

```
df_emp.selectExpr("first_name", "month(to_date(hire_date)) AS  
hire_month").show(5)
```

137. Extract hire day.

LinkedIn@AshishZope

```
df_emp.selectExpr("first_name", "day(to_date(hire_date)) AS hire_day").show(5)
```

138. Calculate employee experience in days.

LinkedIn@AshishZope

```
df_emp.withColumn("experience_days", datediff(current_date(),
to_date(col("hire_date")))).select("first_name", "hire_date",
"experience_days").show(5)
```

139. Calculate employee experience in years.

LinkedIn@AshishZope

```
df_emp.withColumn("experience_years", floor(months_between(current_date(),
to_date(col("hire_date"))) / 12)).select("first_name", "hire_date",
"experience_years").show(5)
```

140. Categorize employees:

LinkedIn@AshishZope

< 2 years	→ Fresher
2-5 years	→ Experienced
> 5 years	→ Senior

LinkedIn@AshishZope

```
df_emp.withColumn(
    "experience_years",
    floor(months_between(current_date(), to_date(col("hire_date"))) / 12)
).withColumn(
    "experience_category",
    when(col("experience_years") < 2, "Fresher")
    .when((col("experience_years") >= 2) & (col("experience_years") <= 5),
    "Experienced")
    .otherwise("Senior")
).select("first_name", "experience_years", "experience_category").show(10)
```

141. Find employees hired in the last 2 years.

LinkedIn@AshishZope

```
df_emp.filter(months_between(current_date(), to_date(col("hire_date"))) <=
24).show()
```

142. Find employees hired in the current year.

LinkedIn@AshishZope

```
df_emp.filter(year(to_date(col("hire_date"))) == year(current_date())).show()
```

143. Find employees hired in January.

LinkedIn@AshishZope

```
df_emp.filter(month(to_date(col("hire_date"))) == 1).show()
```

144. Count employees hired by year.

LinkedIn@AshishZope

```
df_emp.withColumn("hire_year",
year(to_date(col("hire_date")))).groupBy("hire_year").count().show()
```

145. Count employees hired by month.

LinkedIn@AshishZope

```
df_emp.withColumn("hire_month",
month(to_date(col("hire_date")))).groupBy("hire_month").count().show()
```

146. Find the earliest hire date.

LinkedIn@AshishZope

```
df_emp.agg(min(to_date(col("hire_date"))).alias("earliest_hire_date")).show()
```

147. Find the latest hire date.

LinkedIn@AshishZope

```
df_emp.agg(max(to_date(col("hire_date")))).alias("latest_hire_date")).show()
```

148. Find employees with birthdays in a particular month.

LinkedIn@AshishZope

```
df_emp.filter(month(to_date(col("date_of_birth"))) == 5).show()
```

149. Calculate age from date of birth.

LinkedIn@AshishZope

```
df_emp.withColumn("age", floor(months_between(current_date(),
to_date(col("date_of_birth"))) / 12)).select("first_name", "date_of_birth",
"age").show(5)
```

150. Find employees older than 30.

LinkedIn@AshishZope

```
df_emp.withColumn("age", floor(months_between(current_date(),
to_date(col("date_of_birth"))) / 12)).filter(col("age") > 30).show()
```

DATA ENGINEER

● LEVEL 12 — JOIN Operations

Load:

LinkedIn@AshishZope

```
df_emp
df_dept
df_jobs
```

151. Inner join employees with departments.

LinkedIn@AshishZope

```
df_emp.join(df_dept, "department_id", "inner").show()
```

152. Add department name to employees.

LinkedIn@AshishZope

```
df_emp.join(df_dept.select("department_id", "department_name"),  
"department_id").show()
```

153. Add department location to employees.

LinkedIn@AshishZope

```
df_emp.join(df_dept.select("department_id", "location"),  
"department_id").show()
```

154. Join employees with jobs.

LinkedIn@AshishZope

```
df_emp.join(df_jobs, "job_id", "inner").show()
```

155. Add job title to employees.

LinkedIn@AshishZope

```
df_emp.join(df_jobs.select("job_id", "job_title"), "job_id").show()
```

156. Add job level to employees.

LinkedIn@AshishZope

```
df_emp.join(df_jobs.select("job_id", "job_level"), "job_id").show()
```

157. Add minimum and maximum salary for the job.

LinkedIn@AshishZope

```
df_emp.join(df_jobs.select("job_id", "min_salary", "max_salary"),
"job_id").show()
```

158. Join employees + departments + jobs.

LinkedIn@AshishZope

```
df_emp.join(df_dept, "department_id", "inner").join(df_jobs, "job_id",
"inner").show()
```

159. Find employees whose salary is outside their job's salary range.

LinkedIn@AshishZope

```
df_emp.join(df_jobs.select("job_id", "min_salary", "max_salary"),
"job_id").filter((col("salary") < col("min_salary")) | (col("salary") >
col("max_salary"))).show()
```

160. Find employees earning above the maximum salary of their job.

LinkedIn@AshishZope

```
df_emp.join(df_jobs.select("job_id", "max_salary"),
"job_id").filter(col("salary") > col("max_salary")).show()
```

161. Find employees earning below the minimum salary of their job.

LinkedIn@AshishZope

```
df_emp.join(df_jobs.select("job_id", "min_salary"),
"job_id").filter(col("salary") < col("min_salary")).show()
```

162. Find departments having no employees.

LinkedIn@AshishZope

```
df_dept.join(df_emp, "department_id", "left_anti").show()
```

163. Find employees whose department doesn't exist in the department table.

LinkedIn@AshishZope

```
df_emp.join(df_dept.select("department_id"), "department_id",  
"left_anti").show()
```

164. Find employees whose job doesn't exist in the jobs table.

LinkedIn@AshishZope

```
df_emp.join(df_jobs.select("job_id"), "job_id", "left_anti").show()
```

165. Perform a left join between employees and departments.

LinkedIn@AshishZope

```
df_emp.join(df_dept, "department_id", "left").show()
```

166. Perform a right join.

LinkedIn@AshishZope

```
df_emp.join(df_dept, "department_id", "right").show()
```

167. Perform a full outer join.

LinkedIn@AshishZope

```
df_emp.join(df_dept, "department_id", "outer").show()
```

168. Perform a cross join and understand the result.

LinkedIn@AshishZope

```
df_emp.crossJoin(df_dept.select("department_id",  
"department_name").limit(5)).show()
```


● LEVEL 13 — Advanced JOIN Scenarios

169. Find employees and their department heads.

LinkedIn@AshishZope

```
df_emp.join(df_dept.select("department_id", "department_name",
    "department_head_id"), "department_id") \
    .join(df_emp.alias("mgr"), col("department_head_id") ==
col("mgr.employee_id"), "left") \
    .select("employee_id", "first_name", "department_name",
col("mgr.first_name").alias("department_head")).show()
```

170. Find employees and their managers using a self join.

LinkedIn@AshishZope

```
df_emp.alias("e").join(df_emp.alias("m"), col("e.manager_id") ==
col("m.employee_id"), "left") \
    .select("e.employee_id", "e.first_name",
col("m.first_name").alias("manager_first_name")).show()
```

171. Find employees who report directly to each manager.

LinkedIn@AshishZope

```
df_emp.alias("e").join(df_emp.alias("m"), col("e.manager_id") ==
col("m.employee_id"), "inner") \
    .select(col("m.employee_id").alias("manager_id"),
col("m.first_name").alias("manager_name"),
col("e.first_name").alias("direct_report")).show()
```

172. Find managers who have more than 5 employees.

LinkedIn@AshishZope

```
df_emp.groupBy("manager_id").count().filter(col("count") > 5).show()
```

173. Find employees who don't have a manager.

LinkedIn@AshishZope

```
df_emp.filter(col("manager_id").isNull()).show()
```

174. Find departments without a department head.

LinkedIn@AshishZope

```
df_dept.filter(col("department_head_id").isNull()).show()
```

175. Find employees working on projects.

LinkedIn@AshishZope

```
# assuming df_project_members or similar
# df_emp.join(df_project_members, "employee_id", "inner").show()
```

176. Find employees who have no project.

LinkedIn@AshishZope

```
# df_emp.join(df_project_members, "employee_id", "left_anti").show()
```

177. Find employees assigned to multiple projects.

LinkedIn@AshishZope

```
# df_project_members.groupBy("employee_id").count().filter(col("count") >
1).show()
```

178. Find the number of projects per employee.

LinkedIn@AshishZope

```
# df_project_members.groupBy("employee_id").count().show()
```

179. Find the number of employees per project.

```
# df_project_members.groupBy("project_id").count().show()
```

180. Find the department with the most projects.

```
# df_project_members.join(df_emp,  
"employee_id").groupBy("department_id").count().orderBy(col("count").desc()).l  
imit(1).show()
```



LEVEL 14 — Window Functions

181. Rank employees by salary.

```
from pyspark.sql.window import Window  
from pyspark.sql.functions import dense_rank, rank, row_number  
  
window_spec = Window.orderBy(col("salary").desc())  
df_emp.withColumn("salary_rank",  
rank().over(window_spec)).select("first_name", "salary", "salary_rank").show()
```

182. Rank employees by salary within each department.

```
window_spec =  
Window.partitionBy("department_id").orderBy(col("salary").desc())  
df_emp.withColumn("dept_salary_rank",  
rank().over(window_spec)).select("department_id", "first_name", "salary",  
"dept_salary_rank").show()
```

183. Find top 3 employees in each department.

LinkedIn@AshishZope

```

window_spec =
Window.partitionBy("department_id").orderBy(col("salary").desc())
df_emp.withColumn("rn", row_number().over(window_spec)).filter(col("rn") <=
3).drop("rn").show()

```

184. Find highest-paid employee in each department.

LinkedIn@AshishZope

```

window_spec =
Window.partitionBy("department_id").orderBy(col("salary").desc())
df_emp.withColumn("rn", row_number().over(window_spec)).filter(col("rn") ==
1).drop("rn").show()

```

185. Find second-highest-paid employee in each department.

LinkedIn@AshishZope

```

window_spec =
Window.partitionBy("department_id").orderBy(col("salary").desc())
df_emp.withColumn("rn", row_number().over(window_spec)).filter(col("rn") ==
2).drop("rn").show()

```

186. Find third-highest-paid employee in each department.

LinkedIn@AshishZope

```

window_spec =
Window.partitionBy("department_id").orderBy(col("salary").desc())
df_emp.withColumn("rn", row_number().over(window_spec)).filter(col("rn") ==
3).drop("rn").show()

```

187. Use `row_number()` to assign numbers to employees.

LinkedIn@AshishZope

```

window_spec = Window.orderBy(col("salary").desc())
df_emp.withColumn("rn", row_number().over(window_spec)).select("first_name",
"salary", "rn").show(10)

```

188. Use `rank()` on salary.

LinkedIn@AshishZope

```

window_spec = Window.orderBy(col("salary").desc())
df_emp.withColumn("salary_rank",
rank().over(window_spec)).select("first_name", "salary",
"salary_rank").show(10)

```

189. Use `dense_rank()` on salary.

LinkedIn@AshishZope

```

window_spec = Window.orderBy(col("salary").desc())
df_emp.withColumn("salary_dense_rank",
dense_rank().over(window_spec)).select("first_name", "salary",
"salary_dense_rank").show(10)

```

190. Compare `row_number`, `rank`, and `dense_rank`.

LinkedIn@AshishZope

```

window_spec = Window.orderBy(col("salary").desc())
df_emp.withColumn("row_num", row_number().over(window_spec)) \
    .withColumn("rank_val", rank().over(window_spec)) \
    .withColumn("dense_rank_val", dense_rank().over(window_spec)) \
    .select("first_name", "salary", "row_num", "rank_val",
"dense_rank_val").show(10)

```

191. Find employees whose salary is above department average.

LinkedIn@AshishZope

```

dep_avg =
df_emp.groupBy("department_id").agg(avg("salary").alias("dept_avg_salary"))
df_emp.join(dep_avg, "department_id").filter(col("salary") >
col("dept_avg_salary")).show()

```

192. Find employees whose salary is below department average.

LinkedIn@AshishZope

```

dep_avg =
df_emp.groupBy("department_id").agg(avg("salary").alias("dept_avg_salary"))
df_emp.join(dep_avg, "department_id").filter(col("salary") <
col("dept_avg_salary")).show()

```

193. Calculate department average salary for every employee.

LinkedIn@AshishZope

```
dep_avg =
df_emp.groupBy("department_id").agg(avg("salary").alias("dept_avg_salary"))
df_emp.join(dep_avg, "department_id").select("first_name", "department_id",
"salary", "dept_avg_salary").show()
```

194. Calculate salary difference from department average.

LinkedIn@AshishZope

```
dep_avg =
df_emp.groupBy("department_id").agg(avg("salary").alias("dept_avg_salary"))
df_emp.join(dep_avg, "department_id").withColumn("salary_diff_from_avg",
col("salary") - col("dept_avg_salary")).select("first_name", "department_id",
"salary", "dept_avg_salary", "salary_diff_from_avg").show()
```

195. Calculate salary percentage compared to department total.

LinkedIn@AshishZope

```
dep_total =
df_emp.groupBy("department_id").agg(sum("salary").alias("dept_total_salary"))
df_emp.join(dep_total, "department_id").withColumn("salary_pct_of_dept_total",
(col("salary") / col("dept_total_salary") * 100)).select("first_name",
"department_id", "salary", "dept_total_salary",
"salary_pct_of_dept_total").show()
```

**LEVEL 15 — LAG / LEAD**Using `salary_history`:**196. Find previous salary for each employee.**

```
from pyspark.sql.window import Window
window_spec = Window.partitionBy("employee_id").orderBy("effective_date")
df_salary_history.withColumn("prev_salary",
lag("salary").over(window_spec)).show()
```

197. Find next salary for each employee.

```
window_spec = Window.partitionBy("employee_id").orderBy("effective_date")
df_salary_history.withColumn("next_salary",
lead("salary").over(window_spec)).show()
```

198. Calculate salary increment amount.

```
window_spec = Window.partitionBy("employee_id").orderBy("effective_date")
df_salary_history.withColumn("prev_salary", lag("salary").over(window_spec)) \
    .withColumn("increment_amount", col("salary") - col("prev_salary")).show()
```

199. Calculate salary increment percentage.

```
window_spec = Window.partitionBy("employee_id").orderBy("effective_date")
df_salary_history.withColumn("prev_salary", lag("salary").over(window_spec)) \
    .withColumn("increment_pct", when(col("prev_salary").isNull(),
0).otherwise((col("salary") - col("prev_salary")) / col("prev_salary") *
100)).show()
```

200. Find employees whose salary increased by more than 10%.

```
window_spec = Window.partitionBy("employee_id").orderBy("effective_date")
df_salary_history.withColumn("prev_salary", lag("salary").over(window_spec)) \
    .withColumn("pct_change", when(col("prev_salary").isNull(),
0).otherwise((col("salary") - col("prev_salary")) / col("prev_salary") * 100)) \
    .filter(col("pct_change") > 10).show()
```

201. Find employees whose salary decreased.

LinkedIn@AshishZope

```

window_spec = Window.partitionBy("employee_id").orderBy("effective_date")
df_salary_history.withColumn("prev_salary", lag("salary").over(window_spec)) \
    .filter(col("salary") < col("prev_salary")).show()

```

202. Find the first salary record for each employee.

LinkedIn@AshishZope

```

window_spec = Window.partitionBy("employee_id").orderBy("effective_date")
df_salary_history.withColumn("rn",
    row_number().over(window_spec)).filter(col("rn") == 1).drop("rn").show()

```

203. Find the latest salary record for each employee.

LinkedIn@AshishZope

```

window_spec =
    Window.partitionBy("employee_id").orderBy(col("effective_date").desc())
df_salary_history.withColumn("rn",
    row_number().over(window_spec)).filter(col("rn") == 1).drop("rn").show()

```

204. Find the number of salary changes per employee.

LinkedIn@AshishZope

```

window_spec = Window.partitionBy("employee_id").orderBy("effective_date")
df_salary_history.withColumn("prev_salary", lag("salary").over(window_spec)) \
    .filter(col("prev_salary").isNull() & (col("salary") !=
    col("prev_salary"))) \
    .groupBy("employee_id").count().show()

```

205. Find the maximum salary ever received by each employee.

LinkedIn@AshishZope

```

df_salary_history.groupBy("employee_id").agg(max("salary").alias("max_salary_e
ver")).show()

```


● LEVEL 16 — Attendance Practice

Using:

```
df_attendance
```

LinkedIn@AshishZope

206. Find all absent employees.

```
df_attendance.filter(col("status") == "Absent").show()
```

LinkedIn@AshishZope

207. Find all employees who worked from home.

```
df_attendance.filter(col("status") == "WFH").show()
```

LinkedIn@AshishZope

208. Count Present/Absent/WFH/Leave.

```
df_attendance.groupBy("status").count().show()
```

LinkedIn@AshishZope

209. Find attendance count by employee.

```
df_attendance.groupBy("employee_id").count().show()
```

LinkedIn@AshishZope

210. Find absent count by employee.

```
df_attendance.filter(col("status") ==
"Absent").groupBy("employee_id").count().show()
```

211. Find WFH count by employee.

```
df_attendance.filter(col("status") ==
"WFH").groupBy("employee_id").count().show()
```

212. Calculate attendance percentage.

```
attendance_summary = df_attendance.groupBy("employee_id").agg(
    count(when(col("status") == "Present", 1)).alias("present_days"),
    count("*").alias("total_days")
)
attendance_summary.withColumn("attendance_pct", col("present_days") /
col("total_days") * 100).show()
```

213. Find employees with attendance below 80%.

```
attendance_summary = df_attendance.groupBy("employee_id").agg(
    count(when(col("status") == "Present", 1)).alias("present_days"),
    count("*").alias("total_days")
).withColumn("attendance_pct", col("present_days") / col("total_days") * 100)
attendance_summary.filter(col("attendance_pct") < 80).show()
```

214. Find employees with more than 2 absences.

```
df_attendance.filter(col("status") ==
"Absent").groupBy("employee_id").count().filter(col("count") > 2).show()
```

215. Find department-wise attendance percentage.

LinkedIn@AshishZope

```
df_attendance.join(df_emp.select("employee_id", "department_id"),
"employee_id").groupBy("department_id").agg(
    count(when(col("status") == "Present", 1)).alias("present_days"),
    count("*").alias("total_days")
).withColumn("attendance_pct", col("present_days") / col("total_days") *
100).show()
```

216. Find the employee with the highest absence count.

LinkedIn@AshishZope

```
df_attendance.filter(col("status") ==
"Absent").groupBy("employee_id").count().orderBy(col("count").desc()).limit(1)
.show()
```

217. Find the employee with the highest WFH count.

LinkedIn@AshishZope

```
df_attendance.filter(col("status") ==
"WFH").groupBy("employee_id").count().orderBy(col("count").desc()).limit(1).sh
ow()
```

218. Calculate working hours:

LinkedIn@AshishZope

```
check_out - check_in
```

LinkedIn@AshishZope

```
df_attendance.withColumn("working_hours", col("check_out") -
col("check_in")).show()
```

219. Find employees who worked more than 9 hours.

LinkedIn@AshishZope

```
df_attendance.withColumn("working_hours", col("check_out") -
col("check_in")).filter(col("working_hours") > 9).show()
```

220. Find employees who arrived after 9:15 AM.

LinkedIn@AshishZope

```
df_attendance.filter(col("check_in") > "09:15:00").show()
```



● LEVEL 17 — Data Quality

221. Check duplicate employee IDs.

LinkedIn@AshishZope

```
df_emp.groupBy("employee_id").count().filter(col("count") > 1).show()
```

222. Check NULL employee IDs.

LinkedIn@AshishZope

```
df_emp.filter(col("employee_id").isNull()).show()
```

223. Check NULL salaries.

LinkedIn@AshishZope

```
df_emp.filter(col("salary").isNull()).show()
```

224. Find negative salaries.

LinkedIn@AshishZope

```
df_emp.filter(col("salary") < 0).show()
```

225. Find employees with salary = 0.

LinkedIn@AshishZope

```
df_emp.filter(col("salary") == 0).show()
```

226. Validate email format.

LinkedIn@AshishZope

```
df_emp.filter(~col("email").rlike("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$")).show()
```

227. Find duplicate emails.

LinkedIn@AshishZope

```
df_emp.groupBy("email").count().filter(col("count") > 1).show()
```

228. Find future hire dates.

LinkedIn@AshishZope

```
df_emp.filter(to_date(col("hire_date")) > current_date()).show()
```

229. Find employees whose DOB is after hire date.

LinkedIn@AshishZope

```
df_emp.filter(to_date(col("date_of_birth")) > to_date(col("hire_date"))).show()
```

230. Find employees whose salary is outside job salary range.

LinkedIn@AshishZope

```
df_emp.join(df_jobs.select("job_id", "min_salary", "max_salary"), "job_id").filter((col("salary") < col("min_salary")) | (col("salary") > col("max_salary"))).show()
```

231. Check invalid department IDs.

LinkedIn@AshishZope

```
df_emp.join(df_dept.select("department_id").distinct(), "department_id",
"left_anti").show()
```

232. Check invalid job IDs.

LinkedIn@AshishZope

```
df_emp.join(df_jobs.select("job_id").distinct(), "job_id", "left_anti").show()
```

233. Create a data-quality status column:

LinkedIn@AshishZope

Valid
Invalid

LinkedIn@AshishZope

```
df_emp.withColumn(
    "dq_status",
    when(col("employee_id").isNull() | col("salary").isNull() |
col("email").isNull(), "Invalid")
    .otherwise("Valid")
).show()
```

234. Create separate DataFrames for valid and invalid records.

LinkedIn@AshishZope

```
valid_df = df_emp.filter(col("employee_id").isNotNull() &
col("salary").isNotNull() & col("email").isNotNull())
invalid_df = df_emp.filter(col("employee_id").isNull() |
col("salary").isNull() | col("email").isNull())
```

235. Count invalid records by validation rule.

```
df_emp.withColumn(
  "missing_employee_id",
  col("employee_id").isNull()
).withColumn(
  "missing_salary",
  col("salary").isNull()
).withColumn(
  "missing_email",
  col("email").isNull()
).select(
  sum(when(col("missing_employee_id"),
1).otherwise(0)).alias("invalid_employee_id"),
  sum(when(col("missing_salary"), 1).otherwise(0)).alias("invalid_salary"),
  sum(when(col("missing_email"), 1).otherwise(0)).alias("invalid_email")
).show()
```

● LEVEL 18 — Date + Business Scenarios

236. Find employees hired in the last 1 year.

```
df_emp.filter(months_between(current_date(), to_date(col("hire_date"))) <=
12).show()
```

237. Find employees with more than 5 years experience.

```
df_emp.withColumn("experience_years", floor(months_between(current_date(),
to_date(col("hire_date"))) / 12)).filter(col("experience_years") > 5).show()
```

238. Find employees completing 1 year this month.

LinkedIn@AshishZope

```
df_emp.withColumn("months_since_hire", floor(months_between(current_date(),
to_date(col("hire_date")))) \
    .filter(col("months_since_hire") == 12).show()
```

239. Find employees completing 5 years this year.

LinkedIn@AshishZope

```
df_emp.withColumn("experience_years", floor(months_between(current_date(),
to_date(col("hire_date"))) / 12)) \
    .filter(col("experience_years") == 5).show()
```

240. Calculate employee age.

LinkedIn@AshishZope

```
df_emp.withColumn("age", floor(months_between(current_date(),
to_date(col("date_of_birth"))) / 12)).show()
```

241. Calculate employee tenure.

LinkedIn@AshishZope

```
df_emp.withColumn("tenure_years", floor(months_between(current_date(),
to_date(col("hire_date"))) / 12)).show()
```

242. Find average tenure by department.

LinkedIn@AshishZope

```
df_emp.withColumn("tenure_years", floor(months_between(current_date(),
to_date(col("hire_date"))) / 12)) \

.groupBy("department_id").agg(avg("tenure_years").alias("avg_tenure_years")).s
how()
```

243. Find oldest employee in each department.

LinkedIn@AshishZope

```
df_emp.withColumn("age", floor(months_between(current_date(),
to_date(col("date_of_birth"))) / 12)) \
.orderBy(col("department_id"), col("age").desc()).show()
```

244. Find newest employee in each department.

LinkedIn@AshishZope

```
df_emp.orderBy(col("department_id"), col("hire_date").desc()).show()
```

245. Find monthly hiring trends.

LinkedIn@AshishZope

```
df_emp.groupBy(month(to_date(col("hire_date"))).alias("hire_month")).count().o
rderBy("hire_month").show()
```

● LEVEL 19 — Delta Lake

246. Save employee DataFrame as a Delta table.

LinkedIn@AshishZope

```
df_emp.write.format("delta").mode("overwrite").saveAsTable("employee_delta")
```

247. Read the Delta table.

LinkedIn@AshishZope

```
spark.read.table("employee_delta").show()
```

248. Append new employee records.

LinkedIn@AshishZope

```
new_emp_df.write.format("delta").mode("append").saveAsTable("employee_delta")
```

249. Overwrite the Delta table.

LinkedIn@AshishZope

```
df_emp.write.format("delta").mode("overwrite").saveAsTable("employee_delta")
```

250. Update employee salary using Delta.

LinkedIn@AshishZope

```
from delta.tables import DeltaTable

delta_table = DeltaTable.forName(spark, "employee_delta")
delta_table.update(
    condition="employee_id == 101",
    set={"salary": "1200000"}
)
```

251. Delete inactive employees.

LinkedIn@AshishZope

```
delta_table.delete(condition="employment_status = 'Inactive'")
```

252. Perform Delta MERGE.

LinkedIn@AshishZope

```
# Example: merge updates from a staging dataset
# delta_table.alias("t").merge(
#     updates_df.alias("s"),
#     "t.employee_id = s.employee_id"
# ).whenMatchedThenUpdate(set={
#     "salary": "s.salary"
# }).whenNotMatchedThenInsert(values={
#     "employee_id": "s.employee_id",
#     "first_name": "s.first_name",
#     "salary": "s.salary"
# }).execute()
```

253. Implement insert + update using MERGE.

LinkedIn@AshishZope

```
# delta_table.alias("t").merge(
#     staging_df.alias("s"),
#     "t.employee_id = s.employee_id"
# ).whenMatchedThenUpdate(set={"salary": "s.salary"})
# .whenNotMatchedThenInsert(values={"employee_id": "s.employee_id", "salary":
# "s.salary"})
# .execute()
```

254. Implement SCD Type 1.

LinkedIn@AshishZope

```
# Overwrite history with latest value during merge.
# Use employee_id as business key and update the current row with latest
values.
```

255. Implement SCD Type 2.

LinkedIn@AshishZope

```
# Keep historical rows and add effective dates / is_current flag.
# Example logic: add start_date, end_date, is_current columns and expire old
rows.
```

256. Query Delta table history.

LinkedIn@AshishZope

```
DeltaTable.forName(spark, "employee_delta").history().show(truncate=False)
```

257. Perform Delta time travel.

LinkedIn@AshishZope

```
spark.read.format("delta").option("versionAsOf",
0).table("employee_delta").show()
```

258. Restore a previous version.

LinkedIn@AshishZope

```
# delta_table.restoreToVersion(0)
```

259. Add a new column using schema evolution.

LinkedIn@AshishZope

```
# df_emp.withColumn("employee_tier",  
lit("A")).write.format("delta").mode("append").saveAsTable("employee_delta")
```

260. Optimize a Delta table.

LinkedIn@AshishZope

```
spark.sql("OPTIMIZE employee_delta ZORDER BY (department_id)")
```



LEVEL 20 — PySpark Performance

261. Check DataFrame partitions.

DATA ENGINEER

LinkedIn@AshishZope

```
df_emp.rdd.getNumPartitions()
```

262. Repartition employee data.

LinkedIn@AshishZope

```
df_emp = df_emp.repartition(10)
```

263. Coalesce employee data.

LinkedIn@AshishZope

```
df_emp = df_emp.coalesce(2)
```

264. Compare `repartition()` and `coalesce()`.

LinkedIn@AshishZope

```
# repartition increases partition count; coalesce reduces it without full shuffle.
```

265. Cache a DataFrame.

LinkedIn@AshishZope

```
df_emp.cache()
```

266. Check whether DataFrame is cached.

LinkedIn@AshishZope

```
df_emp.is_cached
```

267. Unpersist a DataFrame.

LinkedIn@AshishZope

```
df_emp.unpersist()
```

268. Use `explain()`.

LinkedIn@AshishZope

```
df_emp.explain()
```

269. Analyze a slow join.

LinkedIn@AshishZope

```
df_emp.join(df_dept, "department_id").explain()
```

270. Broadcast the department table.

LinkedIn@AshishZope

```
from pyspark.sql.functions import broadcast
df_emp.join(broadcast(df_dept), "department_id").show()
```

271. Compare normal join vs broadcast join.

LinkedIn@AshishZope

```
# Normal join: shuffle on keys
# Broadcast join: small table is sent to each executor
```

272. Identify a potential data-skew problem.

LinkedIn@AshishZope

```
# Look for one key with unusually high row count using groupBy and count().
# df_emp.groupBy("department_id").count().orderBy(col("count").desc()).show()
```

273. Handle a skewed join.

LinkedIn@AshishZope

```
# Use salting, repartitioning, or broadcasting small tables.
# Example: df_emp.withColumn("salt", floor(rand() * 10)).join(...)
```

274. Compare built-in functions with a Python UDF.

LinkedIn@AshishZope

```
# Built-in functions are optimized and vectorized.
# Python UDFs are slower because they serialize data to Python.
```

275. Explain why `collect()` can be dangerous.

LinkedIn@AshishZope

```
# collect() brings all rows to the driver; it can cause memory issues on huge datasets.
```



LEVEL 21 — Real Interview Scenarios

276. You receive 10 million employee records every day. Find duplicate records efficiently.

LinkedIn@AshishZope

```
df_emp.groupBy("employee_id", "email").count().filter(col("count") > 1).show()
```

277. Your employee DataFrame contains 5 million rows and department contains only 20 rows. How would you optimize the join?

LinkedIn@AshishZope

```
from pyspark.sql.functions import broadcast
result = df_emp.join(broadcast(df_dept), "department_id")
```

278. A PySpark job that normally takes 10 minutes suddenly takes 45 minutes. What would you investigate?

LinkedIn@AshishZope

```
# Investigate data skew, bad partitioning, expensive UDFs, stale caches,
shuffle-heavy joins, large scans, and long-running tasks.
```

279. Your DataFrame has 2 billion records. How would you avoid collecting data to the driver?

LinkedIn@AshishZope

```
# Use aggregations, filters, and writes to disk, and keep results distributed.
# Avoid df.collect() or df.toPandas().
```

280. One department contains 80% of all records. How would you handle data skew?

LinkedIn@AshishZope

```
# Use salting, repartitioning, or a broadcast join when possible.
```

281. You need to calculate the top 3 salaries per department.

LinkedIn@AshishZope

```
window_spec =
Window.partitionBy("department_id").orderBy(col("salary").desc())
df_emp.withColumn("rn", row_number().over(window_spec)).filter(col("rn") <=
3).show()
```

282. You need the second-highest salary per department, including duplicate salaries.

LinkedIn@AshishZope

```
window_spec =
Window.partitionBy("department_id").orderBy(col("salary").desc())
df_emp.withColumn("rn", row_number().over(window_spec)).filter(col("rn") ==
2).show()
```

283. You receive employee updates daily. Existing employees should be updated and new employees inserted.

LinkedIn@AshishZope

```
# Use Delta MERGE:
# delta_table.alias("t").merge(staging_df.alias("s"), "t.employee_id =
s.employee_id")
# .whenMatchedThenUpdateAll()
# .whenNotMatchedThenInsertAll()
# .execute()
```

284. You need complete employee salary history. Implement SCD Type 2.

LinkedIn@AshishZope

```
# Keep historical versions with effective_from, effective_to, and is_current columns.
```

285. An employee's salary changes from ₹10 lakh to ₹12 lakh. Store both versions.

LinkedIn@AshishZope

```
# Insert a new row with new effective date and mark old row inactive.
```

286. Your source contains NULL values. Define a data-quality framework.

LinkedIn@AshishZope

```
# 1. Identify nulls
# 2. Replace with defaults or null-aware logic
# 3. Validate key fields
# 4. Reject invalid rows and log reasons
```

287. Your source schema changes by adding a new column. How will your Delta pipeline handle it?

LinkedIn@AshishZope

```
# Delta supports schema evolution with mergeSchema or allowAutoMerge when writing.
```

288. Your source sends the same file twice. How will you make your pipeline idempotent?

LinkedIn@AshishZope

```
# Deduplicate on business key, add file hash, or use watermarking/metadata tracking.
```

289. A file contains 1 million good records and 10,000 bad records. How will you separate them?

```
# Use validation rules and split into valid_df and invalid_df.
```

290. Your pipeline fails halfway through processing. How will you restart it safely?

```
# Use idempotent writes, checkpoints, Delta table versioning, and rerun from the last successful checkpoint.
```



LEVEL 22 — End-to-End Employee Project

Now combine everything.

291.

Read all employee-related tables.

```
df_emp = spark.table("employees")
df_dept = spark.table("departments")
df_jobs = spark.table("jobs")
df_attendance = spark.table("attendance")
df_salary_history = spark.table("salary_history")
```

292.

Create an employee master DataFrame containing:

```

employee_id
full_name
email
department
job_title
salary
bonus
city
hire_date
employment_status

```

```

def create_employee_master(df_emp, df_dept, df_jobs):
    emp = df_emp.join(df_dept, "department_id", "left") \
        .join(df_jobs, "job_id", "left")
    emp = emp.withColumn(
        "full_name",
        concat_ws(" ", col("first_name"), col("last_name"))
    ).select(
        "employee_id",
        "full_name",
        "email",
        "department_name".alias("department"),
        "job_title",
        "salary",
        "bonus",
        "city",
        "hire_date",
        "employment_status"
    )
    return emp

employee_master = create_employee_master(df_emp, df_dept, df_jobs)
employee_master.show()

```

293.

Add:

```

experience_years
age
salary_category

```

```

employee_master = employee_master.withColumn(
    "experience_years",
    floor(months_between(current_date(), to_date(col("hire_date")))) / 12)
).withColumn(
    "age",
    floor(months_between(current_date(), to_date(col("date_of_birth")))) / 12)
).withColumn(
    "salary_category",
    when(col("salary") < 700000, "Low")
    .when(col("salary") < 1200000, "Medium")
    .when(col("salary") < 2000000, "High")
    .otherwise("Very High")
)

```

294.

Calculate department-level:

```

employee_count
average_salary
minimum_salary
maximum_salary
total_salary

```

```

department_summary = employee_master.groupBy("department").agg(
    count("*").alias("employee_count"),
    avg("salary").alias("average_salary"),
    min("salary").alias("minimum_salary"),
    max("salary").alias("maximum_salary"),
    sum("salary").alias("total_salary")
)

```

295.

Rank employees by salary within department.

LinkedIn@AshishZope

```

window_spec = Window.partitionBy("department").orderBy(col("salary").desc())
employee_master = employee_master.withColumn("salary_rank",
rank().over(window_spec))

```

296.

Find top 3 employees in each department.

LinkedIn@AshishZope

```

window_spec = Window.partitionBy("department").orderBy(col("salary").desc())
emp_top3 = employee_master.withColumn("rn",
row_number().over(window_spec)).filter(col("rn") <= 3).drop("rn")

```

297.

Find employees earning above their department average.

LinkedIn@AshishZope

```

dep_avg =
employee_master.groupBy("department").agg(avg("salary").alias("dept_avg_salary"))
above_avg = employee_master.join(dep_avg, "department").filter(col("salary") >
col("dept_avg_salary"))

```

298.

Join employee data with attendance.

LinkedIn@AshishZope

```

df_employee_attendance = employee_master.join(df_attendance, "employee_id",
"left")

```

299.

Calculate employee attendance percentage.

LinkedIn@AshishZope

```
attendance_summary = df_attendance.groupBy("employee_id").agg(
    count(when(col("status") == "Present", 1)).alias("present_days"),
    count("*").alias("total_days")
).withColumn("attendance_pct", col("present_days") / col("total_days") * 100)
```

300.

Join employee data with salary history.

LinkedIn@AshishZope

```
df_employee_salary = employee_master.join(df_salary_history, "employee_id",
"left")
```

301.

Find latest salary for every employee.

LinkedIn@AshishZope

```
window_spec =
Window.partitionBy("employee_id").orderBy(col("effective_date").desc())
latest_salary = df_salary_history.withColumn("rn",
row_number().over(window_spec)).filter(col("rn") == 1).drop("rn")
```

302.

Calculate salary growth percentage.

LinkedIn@AshishZope

```
window_spec = Window.partitionBy("employee_id").orderBy("effective_date")
with_history = df_salary_history.withColumn("prev_salary",
lag("salary").over(window_spec)) \
    .withColumn("salary_growth_pct", when(col("prev_salary").isNull(),
0).otherwise((col("salary") - col("prev_salary")) / col("prev_salary") * 100))
```

303.

Find employees whose salary increased by more than 10%.

LinkedIn@AshishZope

```
with_history.filter(col("salary_growth_pct") > 10).show()
```

304.

Join employee + department + job + project data.

LinkedIn@AshishZope

```
# employee_master.join(df_project_members, "employee_id", "left")
# .join(df_projects, "project_id", "left")
```

305.

Create an employee 360° DataFrame.

LinkedIn@AshishZope

```
# Combine employee, department, job, attendance, project, and salary history
data.
```

306.

Perform data-quality checks.

LinkedIn@AshishZope

```
quality_check = employee_master.withColumn("is_valid",
col("employee_id").isNotNull() & col("salary").isNotNull() &
col("email").rlike("^([A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"))
```

307.

Separate valid and invalid records.

LinkedIn@AshishZope

```
valid_records = quality_check.filter(col("is_valid"))
invalid_records = quality_check.filter(~col("is_valid"))
```

308.

Write valid records to a Delta Silver table.

LinkedIn@AshishZope

```
valid_records.write.format("delta").mode("overwrite").saveAsTable("silver_employee_data")
```

309.

Create department-level Gold data.

LinkedIn@AshishZope

```
department_summary.write.format("delta").mode("overwrite").saveAsTable("gold_department_summary")
```

310.

Create employee-level Gold data.

LinkedIn@AshishZope

```
employee_master.write.format("delta").mode("overwrite").saveAsTable("gold_employee_master")
```

311.

Optimize the Gold tables.

LinkedIn@AshishZope

```
spark.sql("OPTIMIZE gold_employee_master ZORDER BY (department)")  
spark.sql("OPTIMIZE gold_department_summary ZORDER BY (department)")
```

312.

Create a final dataset suitable for Power BI.


```
final_pbi_dataset = employee_master.join(attendance_summary, "employee_id",  
"left").join(latest_salary, "employee_id", "left")  
final_pbi_dataset.write.mode("overwrite").format("parquet").save("/tmp/pbi_emp  
loyee_dataset")
```



Your Practice Order

Don't try all 312 randomly.

Follow this exact sequence:

01-15 → DataFrame Basics
16-25 → Select
26-40 → Filter
41-55 → Computed Columns
56-70 → String Functions
71-80 → NULL Handling
81-90 → Sorting
91-100 → Duplicates
101-130 → Aggregations
131-150 → Dates
151-180 → Joins
181-205 → Window Functions
206-220 → Attendance
221-235 → Data Quality
236-245 → Date Scenarios
246-260 → Delta Lake
261-275 → Performance
276-290 → Interview Scenarios
291-312 → End-to-End Project



Most important for you right now

Since you're **starting from scratch**, don't look at the advanced questions yet.

Start with **1 → 100**.

For each question:

Question → write PySpark code → run it in Databricks → display() result → move to next.

When you get stuck, send me **your code**, not just the question. I'll review it and explain **what is wrong + why + the optimized PySpark approach**, so you actually build the skill rather than memorizing solutions.

